llmctl docs/faq

FAQ

Revision: 3 **Last modified:** 2026-10-12T00:00:00Z

This FAQ answers the Edge Cases enumerated in `specs/001-llmctl-completion/spec.md`, one entry per case (closely related cases are grouped). Every entry is labeled honestly:

- **Status: current behavior** — this is what the codebase does today, verified against real source and/or a real passing test in this repo.
- **Status: wired / library only / not built** (cluster entries) — *wired* means the running `llmctld` does it and a test drives it; *library only* means the code and its unit tests exist but nothing in the daemon calls it yet; *not built* means no code. No entry describes design intent as if it worked.

Per the project's hybrid testing approach (fixtures for CI/CD determinism, real models for release gating — see `docs/quickstart.md`), "current behavior" entries below are backed by either a fixture-driven automated test (part of `make test`) or a real-model manual verification step; each entry says which.

Revision 2 (llmctl 3.3.0). The cluster, tenant, API-key and audit entries were re-audited against the code under `llmctld/` and the tests in `llmctld/internal/**` and `llmctld/test/integration/`. The first revision of this FAQ was written while `llmctld` was a scaffold; that is no longer true: the daemon has a Raft node with real multi-process bootstrap, join/leave and leader failover, an HTTP/3 API with mutual TLS, JWT/RBAC, tenants and namespace isolation, API keys, an in-memory hash-chained audit log, KV-cache WAL/checkpoint replication and a health monitor. What did **not** change is the scope of the verification: the cluster has been exercised with real `llmctld` processes on `localhost` (and the CLI against one real node, see llmctld-cluster-tls), **never on a multi-host deployment**. Each entry below says what is wired into the running daemon, what is only a tested library and what does not

exist. To see which `llmctld` source files exist, run `find llmctld -name '*.go' -not -name '*_test.go'`; to run the daemon's tests, `cd llmctld && go test ./...`.

> **The cluster CLI needs a host `curl` that lists the `HTTP3` feature** (`curl --version`). Otherwise `llmctl cluster|tenant|apikey` report `llmctld unreachable`. Verified (2026-10-08) with a real curl 8.22.0 built with ngtcp2/nghttp3 in a private test prefix: the cluster, apikey and tenant suites took their success path. How to get such a curl and what that verification covered: llmctld-cluster-tls.

Constitution, governance and integrations

Is llmctl part of the constitution, and where does it live?

Yes: it is a constitution **hub engine** at `constitution/submodules/llmctl`, consumed by reference and initialised one level only (`recursive_init: false`). It declares itself in a tracked `helix-deps.yaml` (Constitution §11.4.31) and does not nest the constitution or other own-org repositories: it finds `constitution`, `containers` and `superspec` through `lib/resolve_deps.sh` (Constitution §11.4.28(C)). See dependency layout.

What does the constitution require of the models llmctl runs (§11.4.277)?

A deterministic known-answer validation before any decision-bearing use. Run with `LLMCTL_VALIDATE_STRICT=1` and gate on `llmctl require-validated <profile>` (0 pass, 1 wrong answer, 2 could not determine; 75 pressure refusal is reported as 2, 98 is port in use). A wrong answer is a defect to investigate, not something to retry until green. See validation.

How does this relate to HelixQA Vision QA (§11.4.278)?

HelixQA uses a validated Vision model as its eyes. llmctl's `vision` (baseline hardware) and `vision-pro` (workstation) profiles are the tiers a consumer can declare; each verdict must record the model identity. Only `vision` has real-model runs here (validation).

What is the decisions-model layer (§11.4.279) and how do claude_toolkit and llmctl split it?

llmctl provides the decision profiles and the HTTPS decide gateway with a mandatory bearer key (decide gateway). claude_toolkit's JEV integration is the consumer: it registers the endpoint as a `decision` record (never an alias) and offers a permission gate, an MCP tool and a launch environment to its agents. The anchor text is in the constitution; this FAQ describes only what llmctl does.

Hardware, scheduling, and local model lifecycle

What happens when my hardware is below the minimum supported tier?

Status: current behavior.

`lib/catalog.sh` 's `catalog_classify_tier()` classifies the live hardware probe into one of four tiers (`below-minimum < baseline < workstation < datacenter`) using fixed, documented thresholds (e.g. `below-minimum` is anything under 8 cores + 32 GiB RAM). `catalog_plan_json()` then marks a catalog profile `"recommended"` only when it both fits the live memory budgets *and* its minimum tier requirement is met (`d["tier_ok"] and d["fits"]` , `lib/catalog.sh` around line 195). On a host classified `below-minimum` , no catalog profile can satisfy `tier_ok` , so `llmctl plan` reports `tier: below-minimum` with an empty `recommended` list — exactly as spec.md's edge case describes.

Verified by the deterministic fixture-driven suite `tests/test_planner.sh` , which asserts the exact recommended-profile set for the `baseline` and

`workstation` hardware fixtures under `tests/fixtures/hw-*.json` (part of `make test`; no GPU or network required).

What happens when the combined footprint of the models I asked to start exceeds my budgets?

Status: current behavior.

`lib/scheduler.sh`'s `sched_start()` computes each requested profile's RAM and VRAM footprint, sums them against the live budgets (probed free memory minus a 4 GiB RAM headroom / 15% VRAM headroom, minus whatever is already reserved by running profiles), and refuses with the exact shortfall if the combined total would exceed either budget:

```
err "cannot start '${p}': needs ${ram} MiB RAM + ${vram} MiB VRAM, but only $(( ram_budge
```

(`lib/scheduler.sh` line 212). `llmctl auto <capability>` handles the same situation differently — it evicts non-enabled services in LRU order until the requested profile fits, but never evicts a service the operator explicitly `enable`d (documented in `README.md`'s "Safety guarantees" and `lib/scheduler.sh`'s header comment). Verified by `tests/test_scheduler.sh` against the deterministic hardware fixtures (`make test`).

What happens when a downloaded model's sha256 doesn't match?

Status: current behavior.

`lib/download.sh`'s `_dl_verify_file()` computes the sha256 of the downloaded `.part` file and compares it against the catalog's expected value (or a value fetched live from the Hugging Face API when the catalog entry is `null`) **before** the atomic rename into the final model path — see `lib/download.sh` lines 83–166 and README's "Safety guarantees" section. A mismatch calls `err "sha256 mismatch for ${path}"` and the partially-downloaded content never reaches the final path; the verification command, its exit code, and its output are appended as evidence to `~/.local/state/llmctl/verify/<profile>.log`. Verified by `tests/test_download.sh` (part

of `make test`, using synthetic fixtures — no real multi-GB download needed for this check).

What happens when a model service fails to start, or crash-loops because the model can never actually start?

Status: current behavior.

Both failure shapes are handled by the same OS-level mechanism, described once in the README's "Safety guarantees" section:

- **Linux** (`lib/service_linux.sh`): every generated systemd `--user` unit carries `Restart=on-failure` plus a bounded restart budget — `StartLimitBurst=3` within `StartLimitIntervalSec=900` (see `lib/service_linux.sh`; `RestartPreventExitStatus=SIGKILL SIGTERM 98`). Once that burst limit is exceeded within the interval, systemd stops restarting the unit and `llmctl status` reports it as `failed (crash-loop)` with the last captured log line, rather than looping forever (this is the FR-044 crash-loop signal referenced in `lib/service_linux.sh` around line 197). A model that can never actually start — corrupted GGUF, VRAM no longer sufficient, wrong engine flags — surfaces this way instead of restarting indefinitely.
- **macOS** (`lib/service_macos.sh`): launchd's `KeepAlive` plus a widened `ThrottleInterval=60` bound the restart *rate* (launchd has no native give-up-after-N-restarts primitive, so this is documented as a rate bound rather than a total-attempt bound — see the README's Safety guarantees section for the explicit honest caveat).

Verified by `tests/test_services_crashloop.sh` (part of `make test`, using `LLMCTL_DRY_RUN` fixtures — no real crashing model process needed to exercise the generated-unit-file logic).

Logs for a failed service are captured under `~/.local/state/llmctl/logs/`, addressing the more general "service fails to start" edge case as well.

What happens if my CLI agent's config is invalid?

Status: current behavior (documentation-backed).

`llmctl` doesn't validate a CLI agent's own config file — that file belongs to the agent, not to `llmctl`. What `llmctl` guarantees is a stable, documented OpenAI-compatible endpoint per profile (`127.0.0.1:<fixed-port>` , see the port map in `README.md`) and per-agent config examples + a `verify` step for all 7 supported agents in `docs/integrations.md` (e.g. `pi` 's `models.json` / `settings.json` example around line 155, each agent's own `--version` / `--help` verification command). If an agent's config points at the wrong port, has a malformed JSON provider entry, or otherwise doesn't match its own real, current schema, the agent itself reports a connection error (it cannot reach `llmctl` 's server, or `llmctl` 's server rejects a malformed request) — `docs/integrations.md` 's per-agent sections are the troubleshooting reference, matching spec.md's edge-case description verbatim.

What happens when a model prompt/response needs to be deterministic (e.g. for live challenges)?

Status: current behavior, with one honestly-tracked gap.

Server-side determinism (Clarification 2) is real: `lib/scheduler.sh` 's `LLMCTL_SEED` environment variable, when set, appends `--seed <value> --temp 0` to a llama.cpp profile's launch arguments — it is opt-in rather than baked into every default launch, since an always-fixed seed would make ordinary interactive coding sessions identically non-creative (`docs/integrations.md` , "Live-challenge determinism" section).

But determinism is *measured* one layer up, at the full CLI-agent output artifact (transcript/diff/edited files), not the raw model API response — two runs can still differ in non-deterministic fields unrelated to the model's actual answer (timestamps, per-run temp-directory paths, UUIDs, elapsed-time/token-usage reporting). Per-agent normalization filters at `docs/integrations/normalize_<agent>.sh` (one per supported agent, built on shared primitives in `docs/integrations/lib_normalize_common.sh`) strip exactly those known field classes before a byte-identical comparison. Verified by `tests/test_normalize_agents.sh` (part of `make test`) against synthetic fixture pairs under `tests/fixtures/agent_output/<agent>/` .

Honest boundary (stated explicitly in `docs/integrations.md` itself): those fixture pairs are synthetic, not reverse-engineered from a real captured run of

any of the 7 agents against a live `llmctl` server. The live, real-GPU capture proving "two runs of the same fixed prompt through the real agent produce byte-identical output after normalization" is release-gating work per the project's hybrid testing approach, not yet performed. One non-deterministic field class named by FR-048/Clarification 17 — output *ordering* — is not yet addressed by any of the 7 filters, because no real captured transcript exists yet to show whether a given agent's output needs deterministic re-ordering at all.

What happens when two `llmctl` invocations race on the scheduler state file (e.g. cron + interactive user)?

Status: current behavior.

`lib/scheduler.sh`'s `scheduler::with_lock()` wraps every state-reading and state-writing scheduler operation in an exclusive advisory `flock` on `${LLMCTL_RUNTIME_DIR}/.scheduler.lock`. A second concurrent invocation blocks on `flock -x` until the first releases the lock, then re-reads fresh state before proceeding — it never acts on a stale read (`lib/scheduler.sh` lines 56–70, citing this exact scenario as FR-043 in its own comment, matching Clarification 13). Verified by `tests/test_scheduler_lock.sh` (part of `make test`), which drives two concurrent invocations against the same state directory and asserts no lost update.

What happens when a multi-GB model download is interrupted mid-transfer (network drop, disk full, Ctrl+C)?

Status: current behavior.

`lib/download.sh` downloads into a `<file>.part` sidecar using `curl -fL --continue-at - --retry 3 --retry-delay 5 -o "${part}" "${url}"` (line 144). Re-running the download resumes via an HTTP Range request against the existing `.part` file. If the server doesn't support Range requests (curl exit code 33), the stale `.part` file is discarded and the download restarts from byte 0 rather than hard-failing (lines 148–158, documented inline as a deliberate fallback, not a bug). sha256 is verified only once the file is fully reassembled, and the atomic rename into the final path happens only after that verification succeeds (lines 162–167) — matching Clarification 15 exactly. Verified by `tests/test_download_resume.sh` (part of `make test`, using a

synthetic interrupted-transfer fixture — no real multi-GB network transfer needed to exercise the resume logic itself; the real multi-GB download path is exercised manually per `docs/quickstart.md` section 2 as a release-gating step).

What happens when a host is configured for cluster mode but its local `llmctld` daemon is unreachable?

Status: wired (CLI side), covered by `tests/test_cluster_cli_args.sh` and the cluster suites.

`lib/cluster.sh` 's `cluster::require_daemon()` probes `GET /v1/cluster/status` on `LLMCTL_CLUSTER_ENDPOINT` (default `https://127.0.0.1:9443`) over HTTP/3 with mutual TLS. If the probe fails it exits non-zero with `llmctld unreachable at <endpoint> - cluster mode requires the daemon to be running.` plus how to restart it, and states that llmctl never falls back to single-host scheduling silently (Clarification 12). Two other failures are reported separately: unusable TLS material (`cluster TLS material is unusable ...` , never a fallback to an unverified connection) and a daemon that answers with an error status. The same message appears when the host curl lacks HTTP/3 (see the note at the top of this FAQ). Every `llmctl cluster|tenant|apikey` subcommand goes through this guard.

Distributed cluster orchestration (User Story 7)

Scope of what follows. Cluster behaviour below was exercised with real `llmctld` processes on one machine (`llmctld/test/integration/`), never across hosts. Network partitions and real latency were not exercised.

What happens when a cluster node fails?

Status: partly wired. Every node runs a health monitor (`internal/cluster/health.go`) that checks every other node's `/v1/cluster/status` every 10 seconds (`DefaultHealthCheckInterval`) and fires the failure callback once per failure episode. The callback that is **wired in the daemon** re-assigns the

KV-cache replication role: if the failed node was a tenant's primary, a live replica becomes primary (committed through Raft) (`TestMonitor_UnhealthyPrimary_TriggersReassignment` , `TestFailoverState_ReplicationRoleReassignedOnCrash_NewPrimaryForwardsOnward` , `TestFailoverState_KVCacheSurvivesPrimaryKill`). Placement of a **new** model start skips unhealthy nodes (`TestPlace_UnhealthyNodeSkippedInFavorOfHealthy`). **Not built:** automatic restart of the failed node's running models on a healthy node, and any measured "rescheduled within N seconds" figure. `TestFailoverState_AppendLostBeforeForwarding_IsReportedNotHidden` documents that a write lost before it was forwarded is reported, not hidden.

What happens during a network partition?

Status: library only. `internal/cluster/partition.go` (`PartitionWatcher` : a follower whose last contact with the leader is stale reports `Draining()` , a bounded grace period reports `GraceExpired()` , the leader never drains) is unit-tested (`TestPartitionWatcher_*`), but no code in `llmctld` consults it yet: the daemon does not stop accepting requests on a minority partition. What the Raft library itself guarantees applies: a partition without quorum cannot elect a leader or commit writes. No partition was induced in a test.

What happens when a model exceeds a single node's VRAM?

Status: placement planning only. `internal/cluster/sharding.go` (`PlanShards`) selects distinct nodes whose combined capacity covers an even split and refuses when capacity is insufficient or the shard count is below 2 (`TestPlanShards_*`). **Not built:** splitting a model across nodes at run time (activation passing, KV sharding) and any throughput measurement.

What happens when the Raft leader fails?

Status: wired, exercised with real processes on one machine. `TestClusterBootstrap_ThreeRealProcessesElectLeaderWithin5s` asserts a three-node election completes in 5 seconds and

`TestClusterFailover_KillingLeaderElectsNewRealLeaderAmongSurvivors` kills the leader process and asserts a survivor becomes leader. These are same-host timings, not a network SLA.

What happens when a model replica fails its health check?

Status: partly wired. The monitor marks the node unhealthy (see "node fails") and placement avoids it. **Not verified:** that a replacement replica is spawned to keep a configured replica count; no test asserts a target replica count or an availability figure.

What happens during a split-brain scenario?

Status: by Raft design, not separately tested. Leader election and log commit use `hashicorp/raft`, which requires a quorum; a minority partition cannot commit. llmctld adds no split-brain logic of its own and no test partitions a live cluster, so this is the library's property, not a result measured on llmctld.

What happens to an in-flight streaming (SSE) completion during a planned drain or minority-partition drain?

Status: not built. There is no drain path in the daemon (see the partition entry) and no test of an in-flight stream during a drain. Inference traffic does not flow through `llmctld` at all: clients talk to each node's `llama-server` or the decision gateway directly.

Model state persistence & recovery (User Story 8)

Status: partly wired. `internal/replication/` implements an encrypted write-ahead log, checkpoints with restore, a LoRA-adapter copy helper and an engine-cache transfer; the daemon wires the replication routes (`/v1/replication/append`, `checkpoint`, `state`, `lag`, `enginecache`, JWT-protected) and automatic forwarding to replicas. Verified with real processes on

one machine (`TestFailoverState_KVCacheSurvivesPrimaryKill` , `TestFailoverState_AutomaticForwarding_NoManualFanOut` , `TestReplicationHealth_LagVisibleThenClearsOnRecovery` , `TestReplicationRoutes_AppendCheckpointStateRealHTTP3RoundTrip`). What is **not** wired: the engine-cache save-transfer-restore orchestration against a real engine, and the automatic triggers named below.

What happens when a KV cache checkpoint fails, or the WAL grows too large?

Status: library only for the policy; the explicit checkpoint route is wired. `TestCheckpoint_FailureDoesNotLoseWAL_NextAttemptIncludesDelta` shows a failed checkpoint does not lose WAL entries and the next attempt includes the delta. `ShouldCheckpoint` (token interval, default 1000, or time interval, default 30 s) and `ShouldForceCheckpoint` (WAL larger than a configured size) are tested, but nothing in the daemon calls them: a checkpoint happens when a client posts to `/v1/replication/checkpoint` . The recovery targets of the original spec (token-loss and recovery-time bounds) were not measured.

What happens when a LoRA adapter checkpoint fails?

Status: library only. `ReplicateAdapter` copies the adapter file to every sink synchronously and stops at the first sink that fails, returning an error the caller must treat as a failed creation. There is **no** built-in exponential-backoff retry and **no** alert in the code; no daemon route creates adapters through it (it is used by the engine-cache transfer).

Authentication, authorization & multi-tenancy (User Story 9)

Status: wired for the control plane; not on the inference path. JWT issue/validation, RBAC roles, tenants, quotas, namespaces and API keys are implemented and exposed as HTTP/3 routes (`/v1/auth/token` , `/v1/auth/`

`apikeys[/{id}/rotate]`, `/v1/tenants`, `/v1/tenants/{id}/quota`, `/v1/tenants/{id}/models[...]`). The daemon requires a JWT signing key in its environment and refuses to start without one. These checks protect `llmctld`'s own API; inference requests to the engines do **not** pass through `llmctld`, so tenant quotas and API keys are not enforced on chat traffic.

What happens when a tenant exceeds its quota?

Status: stored and viewable; request-rate enforcement is library only. `llmctl tenant quota <name> ...` sets the quota through `PUT /v1/tenants/{id}/quota` (admin role required, `TestSetTenantQuota_RequiresAdminRole`, `TestTenantQuota_ViewAndSet`). `tenancy.Enforcer.AllowRequest` returns "denied, retry after D" when the per-second rate is exceeded (`TestAllowRequest_ExceedingRateQuotaReturnsRetryAfter`) and `Decider.CheckQuota` audits the outcome, but no HTTP route in the daemon calls it, so no `429` / `Retry-After` is produced today and the "enforced within 1 ms" target was not measured.

What happens when an API key is compromised?

Status: wired. `llmctl apikey rotate <key-id>` calls `POST /v1/auth/apikeys/{id}/rotate`; the old value is rejected immediately (`TestStore_RotatedKey_OldValueImmediatelyRejected`); `DELETE /v1/auth/apikeys/{id}` revokes. Both need a JWT with the tenant-manage role (`TestRotateRevokeAPIKey_RequiresTenantManageRole`, `TestAPIKeyLifecycle_CreateRotateRevoke_RequiresJWT`). The audit log records the key operations that go through the decider; it cannot list "every request the key was used for" because requests to engines are not seen by `llmctld`. Keys live in memory in the daemon (see the audit entry for the same caveat).

What happens when a tenant admin tries to access another tenant's models?

Status: wired. A model is visible only to its owning tenant unless explicitly shared (`POST /v1/tenants/{id}/models/{model}/share`); a start/stop/

status request for a model outside the tenant is denied by the namespace check and the denied decision is appended to the audit log (`TestModelNamespace_IsolationAndSharing`, `TestNamespaceIsolation_*`, `TestMultitenancyIsolation_1000ConcurrentRequestsZeroCrossTenantLeakage`, a same-host test with 1000 concurrent requests and no cross-tenant leak).

What happens when the OIDC provider is unavailable, and what happens once the 5-minute fallback cache then expires?

Status: library only. `internal/auth/oidc.go` implements the described behaviour: an ID token verified before stays usable for `oidcFallbackWindow` (5 minutes) after the provider becomes unreachable, then the call fails with `ErrOIDCFallbackExpired`, which a caller maps to `401`. The daemon does not construct an `OIDCAuthenticator` today (no flag or environment variable configures an OIDC provider), so OIDC login is not available; only the local JWT/API-key path is.

What happens if the audit log itself is tampered with (an entry deleted, reordered, or the log truncated)?

Status: wired, in memory only. `internal/audit/log.go` is a hash-chained, append-only log with a periodic anchor record (head hash and entry count) that catches what a chain alone cannot: deletion followed by a recomputed chain, and tail truncation (Constitution §11.4.268). Proven by `TestVerifyChain_CleanChainPasses`, `TestVerifyChain_TamperedMiddleEntryDetected`, `TestVerifyChain_DoesNotCatchDeletionRecomputedForward` (the honest limit of the chain alone), `TestVerifyAgainstAnchor_DetectsWhatChainAloneCannot`, `TestVerifyAgainstAnchor_DetectsTailTruncation`, `TestVerifyAgainstAnchor_CleanLogPasses` (`cd llmctld && go test ./internal/audit/...`). The daemon's decider appends an entry for every token validation, RBAC check, quota check and tenant-boundary check it performs, and exposes `GET /v1/audit/entries` and `GET /v1/audit/verify` (admin role required, `TestAuditVerify_ReportsCleanChain`).

Limits: the log is created with `audit.NewLog()` at start-up and is **not persisted**, so it is empty after a restart and an attacker with access to the daemon's memory is out of scope; `Log.Anchor()` exists but the daemon never records an anchor anywhere, so the anchor-based check (`VerifyAgainstAnchor`) is a library property: the running daemon's `/v1/audit/verify` walks the chain only.

Release automation (User Story 5)

Status for this whole section: wired (scripts), covered by `tests/test_create_release.sh` , `tests/test_preflight_submodules.sh` and `tests/test_release_no_secrets.sh` ; the `v3.1.0` tag exists on all five remotes (codeberg, gitflic, github, gitlab, gitverse; annotated tag `f322aa0` peeling to `2467cc5` , checked with `git ls-remote` on 2026-10-13); GitHub and GitLab also list a `v3.1.0` release object (`gh release list` , `glab release list`), the other three were not checked for release objects (UNKNOWN). The release tooling lives in `scripts/release/` (`create_release.sh` , `preflight_submodules.sh` , `build_archive.sh` , `scan_archive.py`); see release-process.

What happens when `gh release create` succeeds but `glab release create` fails, or vice versa?

Status: current behavior (`scripts/release/create_release.sh`). The release is treated as not published unless both forges succeed. Per-version, per-forge state is recorded, so a re-run skips a forge already recorded as done (`SKIP <forge>: already published (idempotent retry-only-the-failed-forge)`) and retries only the one that failed, without creating a duplicate. Details: release-process.

What happens when a pinned submodule's ref is unreachable at release-packaging time (upstream private/deleted/rate-limited)?

Status: current behavior (`scripts/release/preflight_submodules.sh`). The preflight walks every submodule of this repository (llama.cpp and colibri; constitution/containers/superspec are hub-resolved since 2026-10-21 and are not walked) and fetches the pinned commit from its configured remote. A ref that is confirmed absent hard-fails naming the submodule and the expected SHA (exit 1); a submodule whose remote could not be reached at all is reported as `UNKNOWN` (network/auth problem, not evidence the ref is missing) rather than as unreachable. No archive is built past a failed preflight. This is distinct from `llmctl doctor`, which only confirms that `submodules/llama.cpp` and `submodules/colibri` are initialized in the working tree.

Decision models (`llmctl decide`)

How accurate are the local decision profiles compared to hosted Jev?

Status: current behavior (honest limitation). No shipped local profile matches hosted Jev on knowledge-heavy or multi-hop hard items, and llmctl makes no parity claim. The per-profile numbers (all with source labels, from `models/catalog.json` `desc` fields and `docs/research/jev-ecosystem.md`): `decide-tiny` (Jev-Style 0.8B v3; a verdict-readout model, not servable by the gateway yet) reports 79.2% on 2,000 typed decisions per its model card — **vendor-measured**; `decide` (Mapika decider-4b v2.1) reports JevBench composite 64.13, ranked #1 of 89 — **vendor-measured** (hosted Jev 1.13.0 itself scores 63.29 on JevBench, measured by the independent benchmark repo); `decide-pro` (Rizzo Flow 4B) reports 0.648 vs hosted Jev's 0.727 on the authors' own typed-decisions benchmark — **vendor-measured**, and the Rizzo authors explicitly disclaim Jev parity and advise recalibration on your own data. The reported `confidence` ((n·p_max − 1)/

(n−1), clamped to [0,1]) is the ecosystem shaping convention, **not** a calibrated error probability. See `docs/decision-models.md` "Honest limitations".

Why is StartLux-Decision not one of the default profiles?

Status: current behavior (deliberate exclusion). StartLux-Decision's code is Apache-2.0 but its **weights are CC BY-NC-4.0** (non-commercial), so shipping it as a default catalog profile would impose a non-commercial license on every llmctl user's orchestrator. It is excluded from the shipped catalog; the six shipped profiles are Apache-2.0-weighted except `decide-nli` (MIT-weighted) — all permissive. It remains a user-installable option (add a custom decide-capable profile via your own catalog overlay) at your own license risk — llmctl's mechanism (lettered options + first-token `top_logprobs` over stock llama-server) works with any decision-fine-tuned GGUF llama.cpp can load.

When does `llmctl decide` prompt interactively, and how do I disable it in CI?

Status: current behavior. Non-interactive is the default everywhere; the wizard is the project's single sanctioned interactive exception, activated only by: `llmctl decide interactive` on a TTY; the `--interactive` flag on `decide interactive` or `decide ask` (also allows piped stdin); or bare `llmctl decide` with both stdin and stdout on a TTY. Non-TTY stdin without `--interactive` exits 2 with `interactive mode requires a TTY`. `LLMCTL_DECIDE_NO_INTERACTIVE=1` makes every interactive path exit 2 even with the flag — set it in CI (the test suite's own helpers do). All wizard prompts go to stderr; stdout carries only the final JSON. Verified by `tests/test_decide.sh` §9 (refusal paths, scripted-heredoc happy path, abort path).

How do I run N parallel instances of one decision profile?

Status: current behavior — `llmctl decide scale <profile> <N>`. `llmctl decide capacity` (and the planner's `decision_instances`) is a **read-only capacity report**: it tells you how many instances *would* fit in GPU mode or CPU mode (alternative placements, never additive; `total_decision_slots = max(gpu, cpu) × parallel`), but it reserves and

launches nothing. `llmctl decide scale decide-kev-08b 2` is what starts them:

- Instance keys are `decide-kev-08b`, `decide-kev-08b.2`, `decide-kev-08b.3` … The primary keeps its documented port; every further instance gets a port from the registry allocator (so instances beyond the first need the registry binary: `llmctl build decide`; without it the scale is refused before anything starts).
- Scale-up is **admission-bounded** by the same RAM/VRAM budget check `llmctl start` uses and is **all-or-nothing**: if instance K does not fit, nothing is started and the command exits **3** with the numbers (what the instance needs, what remains, how many more would fit).
- Scale-down stops the **highest-numbered** instances first. Asking for the current count is a no-op.
- A runtime failure while scaling up rolls back every instance started in that call (exit 1).
- After a failed `llmctl switch`, the restore brings back the same **count** of instances of a scaled profile, not necessarily the same keys (if only `decide-kev-08b.3` was running it comes back as `decide-kev-08b`).
- `LLMCTL_DECIDE_MODE=deterministic` (default) serves a profile from its primary and overflows to the next instance only when the primary is saturated; `throughput` spreads least-loaded and marks every response `x-llmctl-decide-mode: throughput`.

Is the decide gateway wire-compatible with the official hosted Jev API?

Status: current behavior — compatible wire shape, not a byte-for-byte clone. The gateway accepts and returns the TypeSafe 1P shapes (`POST /v1/systemone` with `{model, state, questions}`; typed noul/choice/score answers; question names never sent to the model), and the official TypeSafe SDKs work against it unchanged via `TYPESAFE_BASE_URL=https://127.0.0.1:8095` + `TYPESAFE_API_KEY=$LLMCTL_API_KEY` and the local CA (`SSL_CERT_FILE`; verified end to end by `tests/test_gateway_endpoints.sh`). Known, documented differences: `usage.input_tokens` is an estimate (`ceil(rendered chars/4)`, not

tokenizer-exact - hosted Jev meters real tokens); `usage.output_tokens` is exactly one per question for decoder profiles; option caps are 20 (hard cap 26, letters A..Z) for decoder profiles vs hosted Jev's 255 choices (native/encoder profiles use 255); the response `model` is the served profile id; the gateway serves every ready decision profile (`/v1/models` lists them with the aliases `jev-latest` / `jev-preview` / `llmctl-<profile>`); and the state budget is `LLMCTL_DECIDE_MAX_STATE_CHARS` (default 8192) - over-budget is rejected with 422, or head+tail shortened with header `x-llmctl-decide-truncated: true` when `LLMCTL_DECIDE_TRUNCATE=1` - rather than hosted Jev's 64k-token context.

How are the decision models' sha256 pins maintained?

Status: current behavior. All twelve decide profiles ship pinned per-file sha256 + byte size + an immutable `hf_revision` commit hash in `models/catalog.json` (the original three were captured 2026-10-06 from the Hugging Face API via `hf-mirror.com` ; their raw values are archived in `docs/research/decision-model-hashes.md`). There is no live-fetch fallback for these LFS weights — the `null` -sha256 API-lookup escape hatch is only for small non-LFS files. Download verification happens before the atomic rename (a mismatch never lands at the final path, evidence appended to `~/.local/state/llmctl/verify/<profile>.log`), and the re-pin procedure for an intentional upstream change is documented in `docs/decision-models.md` "sha256 pin procedure". Verified structurally by `tests/test_catalog_json.sh` and behaviorally by `tests/test_decide_download.sh` and `tests/test_onnx_download.sh` .

What is the `onnx` engine, and why does `llmctl doctor` only WARN about its dependencies?

Status: current behavior. The `onnx` engine (`lib/onnx_server.py` , added in iteration 2) serves encoder-class decision models — currently the `decide-nli` profile (DeBERTa-v3-large zeroshot NLI, port 8096). It is a minimal internal scoring runtime (Python only because `onnxruntime` is Python-only; typed-question logic lives in the Go gateway). `llmctl build onnx` creates a hash-locked private venv (`--require-hashes`) with `onnxruntime` + `numpy`

(CPU execution), `sentencepiece` (the DeBERTa-class `spm.model` tokenizer) and `tokenizers` (`tokenizer.json` repos). `llmctl doctor` reports missing `onnxruntime` / `sentencepiece` as **WARN, not FAIL**, because every `llama` -engine profile (including `decide` / `decide-pro` / `decide-2b` / `decide-max`) runs without them (whether each of those answers well is a separate matter: see `docs/decision-models.md` , live results), and the runner itself dies with a clear, actionable error at launch when a package is missing. Install with `llmctl build onnx` . Decisions are NLI entailment: premise = state, one hypothesis per option, one encoder forward pass per option (latency grows linearly with option count, hence `decide-nli` 's `ctx: 512` / `parallel: 1` ; CPU-only, RAM reservation = model size × 1.5 + 512 MiB, VRAM 0). Verified by `tests/test_onnx_runtime.sh` (real sockets, stub model backends on `PYTHONPATH`).

Why is there no Laya (`convaiinnovations/laya`) profile, and how do I run one anyway?

Status: current behavior (deliberate non-inclusion + a documented BYO-ONNX path). Laya (ModernBERT-large + decision head, Apache-2.0) ships **no prebuilt ONNX export** in its HF repo — verified 2026-10-06 via the HF API: safetensors only, and no `laya-onnx` sibling repo exists on the hub (ONNX is generated client-side by the `laya[onnx]` pip extra). So llmctl ships no Laya catalog profile. The `onnx` runner is deliberately generic, though — the BYO-ONNX path (full detail in `docs/decision-models.md` "Laya (BYO-ONNX...)"):

1. Export the model to ONNX yourself (e.g. `pip install "laya[onnx]"`).
2. Put `model.onnx` in a model dir with the tokenizer (`tokenizer.json` for Laya's BPE — needs `pip install tokenizers` — or `spm.model`), and a `config.json` whose `id2label` names `entailment` and `contradiction` (or exactly `entailment` and `not_entailment`). Without a usable `id2label` the runtime reports `LABEL_n` names and the gateway refuses to guess a column (`502` , instance degraded).
3. Add a custom `"engine": "onnx"` , `"capability": ["decide"]` profile with real sha256/size pins to a **copied** catalog.

4. Point llmctl at it with `LLMCTL_CATALOG=/path/to/catalog.json` and use `llmctl models download` / `llmctl decide ask` as normal — no code changes needed.

Can I use a decision answer as a safety gate?

Not as the only barrier. A decision model is not a safety guardrail: llmctl has not measured how often a harmful input passes and makes no such claim. Use `--min-confidence` so a weak answer abstains (exit 10), fail closed when the gateway is unreachable, and keep a human or an allow-list in front of destructive actions. See limitations.

The gateway refuses to start with exit 4 after the upgrade. Why?

3.1.0 rejects weak access keys. A key in `.env` made of repeated characters, a repeated block or fewer than about 128 estimated bits is refused (the message names the rule, never the value). Run `llmctl decide key rotate` to replace it with a generated 256-bit key, then restart the gateway and update your clients (runbooks).

I rotated the key but the old one still works.

Your gateway probably takes `LLMCTL_API_KEY` from its own environment (a service unit or supervisor): the rotation changed the key file only and printed a `scope:` line saying so. Change or unset the variable where the gateway gets it and restart the gateway.

Can I reach the gateway from outside my home network?

Yes, but choose how deliberately: a VPN or an SSH tunnel keeps it off the public Internet; a port-forward or reverse proxy needs the public name in the certificate (`LLMCTL_TLS_SAN`) and a firewall or proxy for flood protection. Never disable certificate verification. See cloud-exposure.

`llmctl decide smoke` says "unknown decide subcommand".

Only an installation from before the front-end fix does that: `smoke`, `mcp` and `vantage` are subcommands of the Go binary that the first 3.1.0 candidate did not forward. Current trees forward them (`lib/decide.sh`, `_DECIDE_FORWARDED`), so update; as a workaround run `build/llmctl-decide smoke ...` directly (`LLMCTL_DECIDE_BIN` locates the binary).

Which other tools have "jev" in the name, and are they part of llmctl?

None of them. See related-tools.

Are answers repeatable?

In the default deterministic mode the same request gets byte-identical answers from the same engine instance on the same device placement. A busy primary can overflow to another instance (named in `x-llmctl-decide-instance`), and a CPU and a GPU run may differ in the last digits (limitations).

From another machine, `https://HOST:8095` fails certificate verification although the CA is trusted.

The host in the URL must be in the certificate's SAN list. `llmctl decide cert show` prints `sans`; add the missing name or address with `llmctl decide cert renew --san dns:NAME,ip:ADDR` and then signal the running gateway (`kill -HUP "$(awk '{print $1}' "$LLMCTL_STATE_DIR/decide/gateway.pid")"`), because it keeps serving the old pair until it reloads. This is the nezha experience (`nezha.local` and the LAN address failed until re-issued with SANs); see lan-exposure. A `.local` name can also fail only in llmctl's own client when the resolver does not do mDNS: use the address.

I renewed the certificate and the new name is still refused / my extra names disappeared.

Two separate effects, both observed: the gateway serves the new leaf only after SIGHUP or a restart, and each `cert renew` rebuilds the SAN list from the host plus the `--san` / `LLMCTL_TLS_SAN` of **that** run, so pass the full list every time. A name outside the CA's constraints (a public DNS name) is refused with exit 5; see runbooks.

Does the gateway come back after a reboot or logout?

Only if it is a user service with linger: `llmctl decide serve --enable` installs `llmctl-decide-gateway.service` and runs `loginctl enable-linger`; engines need `llmctl enable <profile>` (a plain `start` does not survive a reboot). persistent-services has the unit, `gateway.conf`, drop-ins and how to verify. A real reboot was not performed on the development host.

A slow CPU request was cut off when I restarted the gateway.

On stop the gateway drains in-flight requests for at most 15 s (`LLMCTL_DECIDE_DRAIN_GRACE`), while the CPU-adaptive request deadline is 120 s; a request still running after the grace is closed. This is a known open gap (G-161) and the behaviour is unchanged; the two values to raise together and the caveat that this was not exercised are in persistent-services.

How do I back up and restore the certificate and the key?

Archive `$LLMCTL_HOME/cert/` and the installation `.env` with owner-only modes, restore them with the same modes, then run `llmctl decide cert doctor` and `llmctl decide key doctor`. Clients keep working as long as the CA is the same. The exercised procedure and the failure you get from a loosened key file are in runbooks.

What does the engine pin mean for upgrade and rollback?

The llama.cpp pin is the submodule commit recorded by the release tag (`b10969` in 3.0.2, `b11379` in 3.1.0). Moving to another tag with `git`

`checkout` plus `git submodule update --init --recursive`, then `llmctl build all` and `llmctl install`, moves the engine with it. Certificates, keys and models are untouched. See runbooks.

Host safety and the model validation gate (release 3.2.0)

Apart from the guarded live runs on this host (`small`, once, 2026-10-14: evidence, PASS, `chat-v1` 6/0/0, `MemoryPeak` 925 MiB; and `vision`: evidence: PASS, 21 pass / 0 fail / 0 undetermined, run 1 did not capture `MemoryPeak`, run 2 did: 3005 MiB on this host; `VmHWM` UNKNOWN; the profiles tested were `vision` and `small` only), none of the entries below has been exercised against a real model or a real systemd user manager under real memory pressure; they describe what the code and the hermetic tests do. No other profile has been validated (`fast` has not run live).

Why was my `start` / `enable` / `switch` refused with exit 75?

The memory-pressure start gate (`hw_pressure_gate`) refused it. Exit 75 is `EX_TEMPFAIL`: nothing was started (a `switch` is refused before anything is stopped, so what was running still runs). The gate refuses when memory PSI `full avg10 > 2`, `some avg10 > 10`, or `MemAvailable` is below the profile's footprint plus 4 GiB. Look at the numbers it printed, check `head -2 /proc/pressure/memory` and `grep MemAvailable /proc/meminfo`, free memory or wait, then retry once. A systemd-initiated restart is gated the same way (first `ExecStartPre` of the unit), so a refused restart leaves the unit failed and the start limit applies. Tuning exists (`LLMCTL_GATE_FULL_MAX`, `LLMCTL_GATE_SOME_MAX`, `LLMCTL_GATE_RESERVE_MB`) but raising it on a host that is really short of memory defeats the purpose. Details: host-safety, architecture.

Why did my service not restart after it was killed?

By design. The unit uses `Restart=on-failure` with `RestartPreventExitStatus=SIGKILL SIGTERM 98`: a process ended by

`SIGKILL` (the kernel OOM killer, `systemd-oomd`, `kill -9`), a deliberate `SIGTERM` stop, or exit 98 (port already in use) is not restarted, because a restart would repeat the same outcome and, for a model load, repeat the memory spike. Other failures restart after a growing delay and stop for good after 3 starts in 900 s (`StartLimitBurst=3`, `StartLimitIntervalSec=900`). Find the cause (`journalctl --user -u 'llmctl-*@<profile>' -b`, `llmctl logs <profile>`), then `llmctl start <profile>`. State diagram: architecture. Whether this systemd applies `RestartPreventExitStatus` to a failing `ExecStartPre` is UNKNOWN; the start limit applies either way.

How do I validate a model?

`llmctl validate <profile>` runs every known-answer suite the catalog names for the profile (`validation.suites`) against the running endpoint; if the profile is downloaded but idle it starts it UNPUBLISHED (loopback, a private port, through the scheduler, so budget and pressure gate apply) and stops it again unless you pass `--keep`. `llmctl vision validate [--profile vision|vision-pro]` runs the golden-screenshot suite only, `llmctl validate-all` covers every running profile, and `llmctl require-validated <profile>` is the check a consumer script uses (exit 0 only for a fresh passing record). Exit codes: **0** every check passed, **1** a wrong result, **2** could not determine. A model that was never run through this is **not validated**, and `llmctl models verify <profile>` and `llmctl validate-all` say so. Reference: validation. The gate is enforced and strict by default: a profile without a fresh, signed PASS record is started unpublished (127.0.0.1, a private port, no registry row), validated, and published on its documented port only on PASS; a wrong **or** undetermined result stops it and the command exits non-zero; a systemd unit without a fresh record is skipped at boot/restart. `LLMCTL_VALIDATE_STRICT=0` (serve an undetermined profile, marked UNVALIDATED) is refused for consumer-facing profiles. Projects should ask `llmctl endpoint <profile>` for the URL: it prints one only for a validated profile.

Why does the first start of a profile take twice as long now?

Because it has no validation record yet: llmctl loads the model once unpublished to validate it, and again on its public port after it passed. Later starts with a fresh record (same model bytes, engine, flags, suites; younger than `LLMCTL_VALIDATION_MAX_AGE`) load it once.

What does UNDETERMINED mean?

Exit 2: the run could not decide, so it is neither a pass nor a fail and is never recorded as a pass. Typical causes: the profile is not downloaded or not served, the endpoint is down, the wrong model answers (identity check against `/v1/models`), the host is under pressure, the token budget ran out, the completion was cut off with reasoning content, the message content was empty, the reply had a malformed shape, or the overall deadline (`LLMCTL_VALIDATE_TIMEOUT`, default 180 s) passed. A truncated answer that would have passed is still UNDETERMINED (a cut-off pass is not trusted); a truncated answer that is already wrong is a FAIL. An UNDETERMINED run never replaces an earlier PASS record. Remedy: fix the cause, raise `request.max_tokens` for a reasoning model, retry. Never loosen a check.

How do I run the safe live test?

Only with the operator's explicit go-ahead, on a quiet host, and once: `scripts/livetest/vision_live_test.sh [--allow-large] [<profile>]` (default `vision`; the profile must be in the catalog, listed as fitting by `llmctl plan --json`, checksum-verified and have a validation suite; large profiles such as `vision-pro` also need `--allow-large`), or `scripts/livetest/run_profiles.sh` for an ordered list, one at a time. It refuses with exit 3 when the profile is not allowed or memory, PSI, VRAM or the running set is not quiet, starts a watchdog that stops the profile through `llmctl stop <profile>` when a limit is crossed (exit 4), runs the profile's validation (`llmctl vision validate` or `llmctl validate`), always stops the profile and checks the VRAM baseline. Evidence goes to `~/.cache/llmctl-livetest/<UTC timestamp>-<profile>/`. Do not wrap it in retries and do not set `LT_HERMETIC` for a live run. Protocol and exit codes: live-test-protocol. It has

run live for `vision` twice on 2026-10-13 (evidence) and for `small` once on 2026-10-14 (evidence); PASS every time, one host; no other profile has run live yet.

Where is the systemd-oomd 50% rule?

It is not an llmctl setting. It is the distribution's vendor drop-in on `user@.service` (`ManagedOOMMemoryPressure=kill` , pressure limit 50% for 20 s), inspect it with `systemctl --user cat user@.service` or `systemctl cat user@.service` . On 2026-10-12 it made `systemd-oomd` kill 259 units in one event, among them `llmctl-llama@vision.service` . llmctl does not change it (that is an operator decision); it reduces its own exposure (per-profile caps, no swap, higher OOM score, `ManagedOOMPreference=avoid` , which may be ignored for an unprivileged unit) and, because `SIGKILL` is not restarted, a unit killed this way stays down. Details: host-safety.

Why did `llmctl doctor` FAIL for two units?

A profile has two enabled engine units (for example `llmctl-llama@decide-nli` and `llmctl-onnx@decide-nli`), or a live engine process of this install has no registry row. Two units for one profile fight over one port and one footprint. `enable` and `disable` now keep exactly one, and `doctor` reports the existing state. llmctl does not remove an existing duplicate for you: list them with `systemctl --user list-unit-files 'llmctl-*@<profile>*'` , run `llmctl disable <profile>` and `llmctl enable <profile>` , and re-run `llmctl doctor` . Do not delete unit files by hand while the unit runs.

Why does the unit have a memory cap I did not set, and how do I change it?

`llmctl install` writes a drop-in `llmctl-<engine>@<profile>.service.d/05-memory.conf` with `MemoryHigh` = 1.25x and `MemoryMax` = 1.5x of the plan footprint, and the template sets `MemorySwapMax=0` . To override, write your own later drop-in (for example `10-hostsafety.conf`) with the values you want, run `systemctl --user daemon-reload` and restart the unit; later

files win. A running unit keeps its old limits until it is restarted. See persistent-services.